

Validate input with serializers



Turn untrusted JSON into clean, checked data with a DRF serializer.

The big idea

Anything a client sends is **untrusted**: a title might be missing, a date might be "next friday", a priority "URGENT!!!". It's like a messy spreadsheet from someone else: check every column before you use it.

A **serializer** is a bouncer with a checklist. Each field is checked on its own, then the whole thing together. You get **clean data** or a dict of **errors**.

What the tests do

The tests create your `TaskSerializer` with some data, then check the result.

```
serializer = TaskSerializer(data={"title": "Write docs", "priority": "low",  
                                "due_date": TOMORROW})  
  
serializer.is_valid()  
serializer.errors           # the test checks this dict  
serializer.validated_data   # and sometimes this one
```

What they expect back

Error messages must match exactly, full stop included.

```
{"title": "Write docs"}
#  valid, and priority becomes "medium"
{"title": "  Deploy  "}
#  valid, and title becomes "Deploy"
{"title": "  ab  "}
#  title: "Title must be at least 3 characters."
{"title": "Write docs", "priority": "urgent"}
#  an error on priority
{"title": "Write docs", "due_date": YESTERDAY}
#  due_date: "Due date cannot be in the past."
{"title": "Fix outage", "priority": "high"}
#  due_date: "High priority tasks need a due date."
```

New words

serializer

A DRF class listing the expected fields and their rules.

is_valid()

Runs every check. Gives **True** or **False**.

validated_data

The clean dict, once **is_valid()** gave **True**.

errors

Field name → list of messages. **{}** when all is fine.

ValidationError

What you **raise** to say "this is not allowed".

attrs

The dict of all field values, passed to **validate()**.

Tool: a Serializer class

Fields are one line each, like a model. Use it with `data=...`, then `.is_valid()`.

```
class PetSerializer(serializers.Serializer):
    name = serializers.CharField(max_length=20)
    age = serializers.IntegerField(min_value=0)

good = PetSerializer(data={"name": " Rex ", "age": "3"})
print(good.is_valid())          # True
print(good.validated_data)      # {'name': 'Rex', 'age': 3}

bad = PetSerializer(data={"age": -1})
print(bad.is_valid())           # False
print(bad.errors["name"][0])    # This field is required.
```

Tools: ChoiceField and DateField

`default=` fills in a missing value. An optional date needs two different options.

```
class ShirtSerializer(serializers.Serializer):
    size = serializers.ChoiceField(choices=["S", "M", "L"], default="M")
    gift_date = serializers.DateField(required=False, allow_null=True)

s = ShirtSerializer(data={"gift_date": None})
s.is_valid()
print(s.validated_data)      # {'size': 'M', 'gift_date': None}

s = ShirtSerializer(data={"size": "XL", "gift_date": "2026-12-24"})
s.is_valid()
print(s.errors["size"][0])   # "XL" is not a valid choice.
```

Tool: `validate_<field>` checks one field

DRF calls it after the field's own checks, with the cleaned value. Raise, or return the value.

```
class UsernameSerializer(serializers.Serializer):
    username = serializers.CharField()

    def validate_username(self, value):
        if value.lower() == "admin":
            raise serializers.ValidationError("That name is reserved.")
        return value.lower()

s = UsernameSerializer(data={"username": " ADMIN "})
print(s.is_valid(), s.errors["username"][0])  # False That name is reserved.
s = UsernameSerializer(data={"username": " Grace "})
print(s.is_valid(), s.validated_data)        # True {'username': 'grace'}
```

Tool: `validate(self, attrs)` checks fields together

Runs last, with all clean values in `attrs`. A dict puts the error on one field.

```
class TripSerializer(serializers.Serializer):
    start = serializers.IntegerField()
    end = serializers.IntegerField()

    def validate(self, attrs):
        if attrs["end"] < attrs["start"]:
            raise serializers.ValidationError(
                {"end": "The trip can't end before it starts."})
        return attrs

s = TripSerializer(data={"start": 10, "end": 4})
print(s.is_valid())          # False
print(s.errors["end"][0])    # The trip can't end before it starts.
```

validate_<field> vs validate

validate_title(self, value)

- Checks **one** field
- Gets that field's clean **value**
- Raise with a plain message
- Must **return value**
- Example: the title is too short

validate(self, attrs)

- Checks **several** fields together
- Gets **attrs**, a dict of all values
- Raise with **{"field": "message"}**
- Must **return attrs**
- Example: high priority needs a date

Pattern: check for None, then compare dates

Dates compare like numbers. `and` stops early, so `None` is never compared.

```
from datetime import date

for value in [None, date(2020, 1, 31), date(2099, 1, 1)]:
    print(value, value is not None and value < date.today())
```

OUTPUT

```
None False
2020-01-31 True
2099-01-01 False
```

Your task, step by step

- 1 Add `priority`: a `ChoiceField` of low, medium, high; default medium → tests 2, 6
- 2 Add `due_date`: a `DateField`, not required **and** allows `None` → tests 1, 7
- 3 `validate_title`: under 3 characters raises; otherwise return it → test 4
- 4 `validate_due_date`: not `None` and before today raises → test 8
- 5 `validate`: high priority with no due date raises on `due_date` → test 9

Common mistakes

- No `return value` or `return attrs` at the end of a check
- Only one of `required=False` and `allow_null=True`: you need both
- `value < date.today()` without checking for `None` first
- A plain string in `validate()`: it lands under `non_field_errors`
- A wrong method name like `validate_titles`: DRF never calls it

Where you'll use this

A to-do app posts a task with the title "ab" and the priority "URGENT". Your API replies **400** with a message for each bad field, and nothing broken is saved.

- In a DRF view: `TaskSerializer(data=request.data)`, then `is_valid()`
- If it fails: `Response(serializer.errors, status=400)`
- Frontend checks are a courtesy; the server's checks protect your data

Check yourself

1. When can you read `serializer.validated_data`?
2. What's the difference between `required=False` and `allow_null=True`?
3. Where does a rule that looks at two fields belong, and why?
4. Why validate on the server if the frontend already checks?

Keep learning

[Tutorial 1: Serialization](#)

Django REST framework · tutorial · beginner. Start here: builds a serializer from scratch and shows `is_valid()`, `validated_data` and errors in the Python shell.

[Use Django REST Framework to Create Web APIs \(video course\)](#)

freeCodeCamp · video · beginner. A free two-hour video that builds a DRF API with serializers, viewsets and tests; handy if you prefer watching.

[Serializers: validation](#)

Django REST framework · docs · intermediate. The official reference for `validate_<field>` and `validate(attrs)`, the two hooks your `TaskSerializer` uses.

[Serializer fields](#)

Django REST framework · docs · intermediate. Lists every field type and option (`required`, `max_length`, `allow_null`) so you can describe the input shape precisely.

[Validators](#)

Django REST framework · docs · intermediate. Reusable validation rules you can attach to fields once your own `validate` methods start repeating themselves.